



Teuton

Test de infraestructura

Agradecimientos

- RubySur
- Asistentes
- Ruby
- Software Libre
- Comunidad Teuton



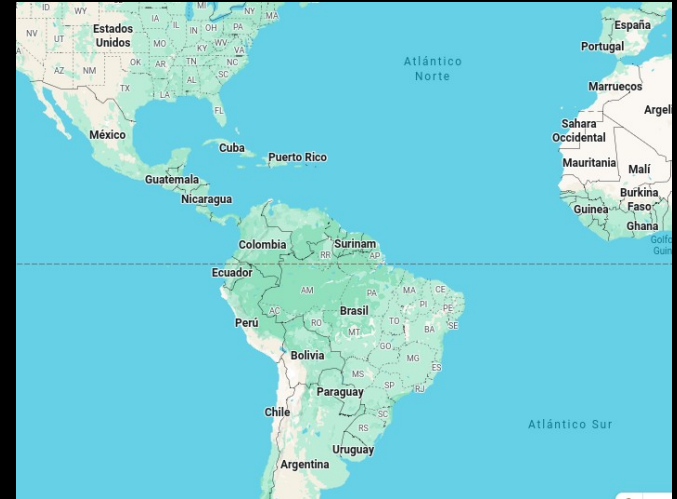
Presentación

David Vargas

- GitHub: @dvarrui
- Email: dvarrui@proton.me
- Telegram: @dvarrui)

Trabajo

- Profesor FP de Sistemas y Aplicaciones Informáticas
- Proyectos Open Source e IA







Presentación

Perfil:

- Ruby
- OpenSUSE
- Software Libre
- Starwars



Agenda

- 1) CONTEXTO del problema
- 2) TEUTON: Test de infraestructura
- 3) **Ruby** es el camino

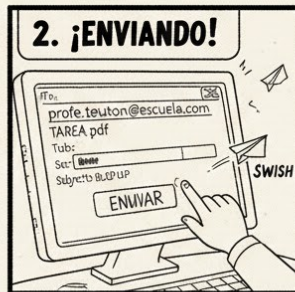


Contexto

- Formación Profesional
- Administración de Sistemas y Redes
- Procesos en educación
- Perfil Ingeniero Informático



EL CICLO DE LA EVALUACIÓN



FIN. HASTA LA PRÓXIMA.



Concurso de programación - PROGRAMAME

[ProgramaMe](#) [Información general](#) [Problemas](#) [Regionales](#) [Nacional](#) [Ediciones anteriores](#) [Contacto](#)

 ProgramaMe: Concurso de Programación para Ciclos Formativos

Estás aquí: / [Nacional 2022](#) / Inicio



Concurso Nacional!

Organizado desde la Facultad de Informática de la UCM

28 de abril de 2022

[Inicio](#)
[Ayuda a Ucrania](#)
[Información para equipos](#)
[Información para centros](#)
[Inscripción](#)
[Reglamento](#)

ProgramaMe en tiempos de pandemia

Ponemos en marcha la final de ProgramaMe con dos elementos a nivel mundial que nos obligan a adaptarnos a las circunstancias:

- *Pandemia de la COVID-19*: la realización de actividades presenciales multitudinarias sigue sin estar completamente despejada, por lo que, como [el curso pasado](#), esta edición tendrá el modelo de *final online multisitio*.
- *Invasión rusa a Ucrania*: el 24 de febrero fuerzas terrestres rusas entraron en Ucrania y desde entonces se vive una crisis humanitaria. Desde ProgramaMe queremos ayudar [Acnur](#), la agencia de la ONU para los refugiados, a [recaudar fondos](#).



[Prueba tus soluciones](#)

Puedes probar tus soluciones en [¡Acepta el reto!](#), el juez on-line con [problemas de ediciones anteriores](#).

[Programame 2.0](#)

Visita nuestro [blog](#) con



Correcciones automáticas basadas en tests

- 🚦 TDD: Desarrollo guiado por pruebas de software.
- 🚦 Lograr código limpio que funcione.
- 🚦 Garantizar que el software cumple con los requisitos que se han establecido.

“Clean Code” (Robert C. Martin)

“Código limpio” (Carlos Ble)



Code testing - Example 1 - Recursive

```
def factorial(n)
  if (n == 0 || n == 1)
    return 1
  else
    return n * factorial(n - 1)
  end
end
```

```
require 'test/unit'
require_relative 'factorial_recursive'

class TestFactorial < Test::Unit::TestCase
  def test_base_case
    assert_equal(1, factorial(0))
    assert_equal(1, factorial(1))
  end

  def test_2_3_4
    assert_equal(2, factorial(2))
    assert_equal(6, factorial(3))
    assert_equal(24, factorial(4))
  end
end
```

Code testing - Example 1 - Recursive

- Los paréntesis son opcionales cuando son redundantes.
- El if admite la forma de lenguaje natural.
- Toda sentencia es una expresión. Esto es, devuelven algo. El último return sobra.

```
def factorial(n)
  return 1 if n == 0 || n == 1
  n * factorial(n - 1)
end
```

```
require 'test/unit'
require_relative 'factorial_recursive'

class TestFactorial < Test::Unit::TestCase
  def test_base_case
    assert_equal(1, factorial(0))
    assert_equal(1, factorial(1))
  end

  def test_2_3_4
    assert_equal(2, factorial(2))
    assert_equal(6, factorial(3))
    assert_equal(24, factorial(4))
  end
end
```

Code testing - Example 2 - Object methods

```
def factorial(n) = (1..n).to_a.reduce(:*)
```

- Una función de una sentencia se puede poner en una línea.
- (1..n) es azúcar sintáctico de Range.new(1,n). Es un objeto.
- Los métodos de los objetos se pueden encadenar. El método to_a devuelve un objeto Array
- El método reduce del Array reduce la lista usando el método indicado.
- Los simbolos existen en muchos lenguajes.

```
require 'test/unit'
require_relative 'factorial_recursive'

class TestFactorial < Test::Unit::TestCase
  def test_base_case
    # assert_equal(1, factorial(0))
    assert_equal(1, factorial(1))
  end

  def test_2_3_4
    assert_equal(2, factorial(2))
    assert_equal(6, factorial(3))
    assert_equal(24, factorial(4))
  end
end
```

Code testing - Example 2 - Object methods

- Esto es lo mismo de antes pero poniendo los métodos de forma más “bonita” o “estilo funcional”.
- reduce devuelve un valor que es el return de la función.

```
def factorial(n)
  (1..n)
    .to_a
    .reduce(:*)
end
```

```
require 'test/unit'
require_relative 'factorial_recursive'

class TestFactorial < Test::Unit::TestCase
  def test_base_case
    # assert_equal(1, factorial(0))
    assert_equal(1, factorial(1))
  end

  def test_2_3_4
    assert_equal(2, factorial(2))
    assert_equal(6, factorial(3))
    assert_equal(24, factorial(4))
  end
end
```

Software Libre / OpenSUSE

- GNU/Linux OpenSUSE, y la comunidad del Software Libre
- Ruby para scripting, YAST, etc.
- **Si te encuentras un problema y lo puedes solucionar... ¡adelante!**



Objetivos

- Reducir tareas repetitivas y “aburridas”
- Procesos reproducibles y automatizados (devops, tests)
- Jugar es divertido (Gamificación)



El camino

Software Libre
GNU/Linux
Terminal CLI
OpenSUSE
TDD / Testing
Ruby
No Rails
Teuton



Empezar a caminar...

- Usar **Ruby** y ser feliz!
- Objetivo: crear una herramienta para facilitar las tareas
- Aprender y aplicar buenas prácticas durante el proceso

Principios:

- TDD (Test-Driven Development)
- DRY (Don't repeat yourself)
- DevOps (Automatización de procesos)



Teuton

Test de Infraestructura



Herramienta para realizar tests de infraestructura



¿Qué es Teuton?

- Programa multiplataforma.
- Licencia Software libre.
- Test de infraestructura.



¿Qué resuelve?

1. Revisar nuestra infraestructura como si fuera código.
2. Automatizar las correcciones de las MV remotas de los alumnos.

Instalación - CLI

- Instalar Ruby
- `gem install teuton`, instalar la gema

```
$ teuton help
```

```
Commands:
```

```
teuton [run] [OPTIONS] DIRECTORY    # Run test from directory
teuton check [OPTIONS] DIRECTORY    # Check test and config file content
teuton config [OPTIONS] DIRECTORY   # Suggest configuration.
teuton help [COMMAND]                # Describe available commands or one specific command
teuton new DIRECTORY                 # Create skeleton for a new project
teuton readme DIRECTORY               # Show README extracted from test contents
teuton version                       # Show the program version
```

RubyGems - “gem install teuton”

Aplicar tests para mantener la calidad de...

- El código
- La infraestructura
- Los scripts
- Configuraciones
- Entornos

¡Cualquier entorno CLI!

The screenshot shows the RubyGems website for the 'teuton' gem. The header includes the RubyGems logo and navigation links: blog, stats, docs, and about. The breadcrumb trail indicates the path: home > teuton > 3.0.0 (latest). The main section displays the gem name 'teuton' in a large font, followed by the version '3.0.0' and the release date 'April 20, 2026'. A button labeled '58 Versions' with a right arrow icon is visible. Below this, the 'TOTAL DOWNLOADS' are shown as '66,060'. For the current version, there are '386' downloads. A 'Star' button with a GitHub icon and the number '36' is also present. On the right side, there are tabs for 'Gem info' (selected) and 'Dependencies'. Under 'Gemfile', the entry 'gem 'teuton', '>= 3.0'' is shown. Under 'Install', the command 'gem install teuton' is displayed. At the bottom right, a description states: 'Infrastructure test, useful for: (1) Sysadmin tea apprentices to evaluate their learning process a machines. Allow us: (a) Write test units for real with requirements on remote machines.'

Dependencias

7 Runtime Dependencies

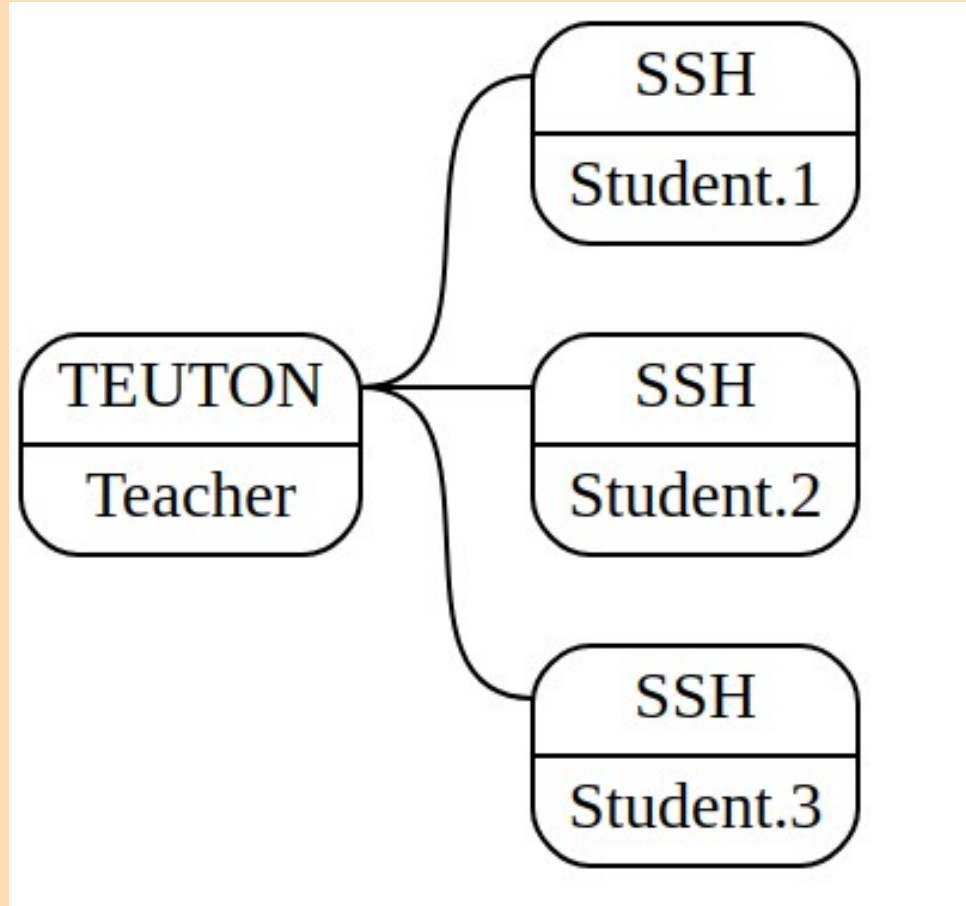
Gem		Required version
json_pure 2.8.1		~> 2.8
net-sftp 4.0.0		~> 4.0
net-ssh 7.3.3		~> 7.3
net-telnet 0.2.0		~> 0.2
rainbow 3.1.1		~> 3.1
terminal-table 4.0.0		~> 4.0
thor 1.5.0		~> 1.3

Entorno de trabajo

- El profesor tiene instalado TEUTON.
- Los alumnos tienen instalado servicio SSH/TELNET.

Un test consiste de:

- 1) **config.yaml**: Fichero de configuración.
- 2) **start.rb**: Tests (target, run ,expect, etc.)



❖ **EVALUACIÓN AUTOMÁTICA CON TEUTON** ❖

1. ¡A PROGRAMAR!



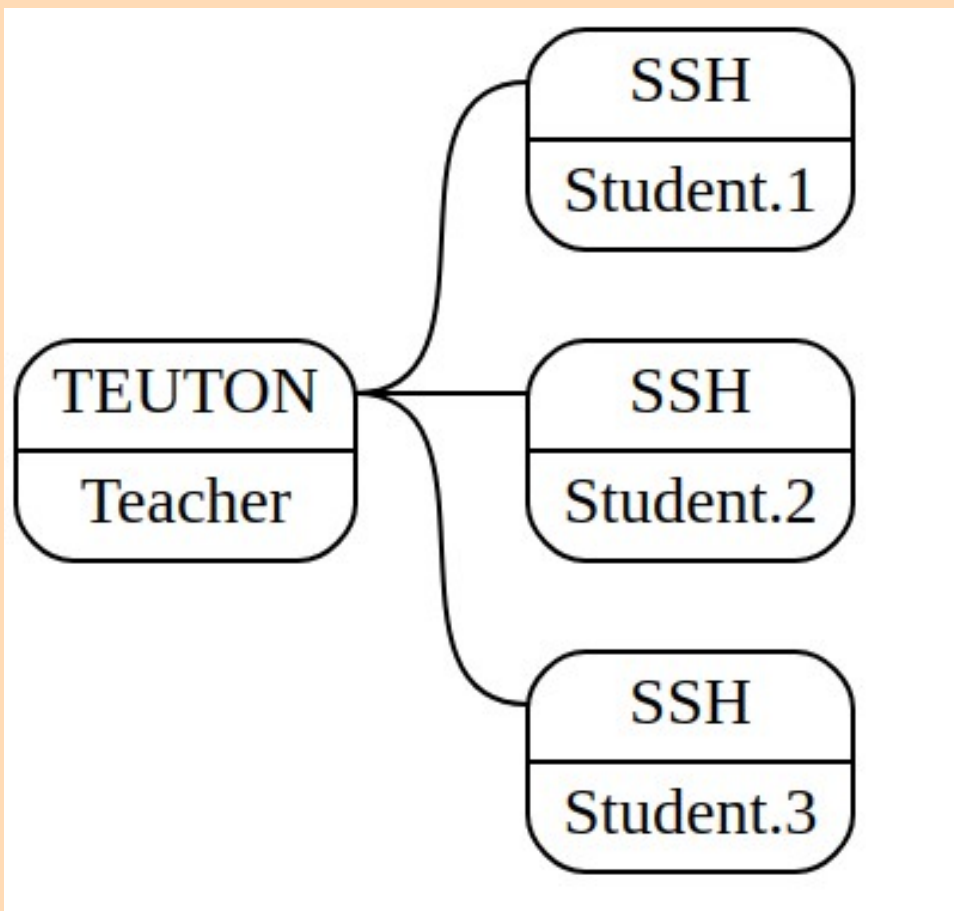
2. ¡AYUDANDO EN CLASE!



3. ¡TEUTON AL RESCATE!



¡EVALUACIÓN RÁPIDA Y JUSTA!



Modos de trabajo

Modo Classroom

- El profesor con TEUTON.
- Los alumnos con SSH.
- Resultados con profesor y/o alumno.

Modo Standalone

- Alumno con TEUTON.
- Resultados con alumno.

Modo Conquest

- Jurado con TEUTON.
- Alumno con SSH.
- Resultados en el Jurado.



Hagamos una
parada...

DSL (Domain Specific Language)

Beneficios:

- Creado para resolver problemas en un campo concreto.
- Permite a los expertos del dominio crear y entender código (modelar problemas del dominio) más fácilmente
- Ejemplos: SQL, CSS, el álgebra en matemáticas. Mayor expresividad y concisión para su dominio específico.

¿Y si creamos un DSL para resolver nuestro problema?

Lenguaje Ruby

- Tipado dinámico
- OO real (todo es un objeto) y de primera clase
- Metaprogramación
- “Parece” lenguaje natural (inglés).
- Legible y fácil de escribir.
- **Creación de DSL (Lenguajes específicos del dominio)**

DSL de Teuton

- **target:** Descripción del objetivo
- **run:** Comando que ejecutamos para comprobar.
- **expect:** Lo que esperamos “ver” en la salida del comando anterior.

```
group "Learn about targets" do
  target "Create user obiwan"
  run "id obiwan"
  expect ["uid=", "(obiwan)", "gid="]

  target "Delete user vader"
  run "id vader"
  expect_fail
end
```

1. Si tienes el comando para comprobar el target: **systemctl status libvirtd**
2. Y sabes lo que se debe "mirar" en la salida: **active (running)**
3. ¡Ya tienes el test!

```
> systemctl status libvirtd
● libvirtd.service - libvirt legacy monolithic daemon
   Loaded: loaded (/usr/lib/systemd/system/libvirtd.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-01-03 19:29:58 WET; 8s ago
     Invocation: 994cbd2a3dd84a9292c104052cd1e017
  TriggeredBy: ● libvirtd.socket
                ● libvirtd-ro.socket
                ● libvirtd-admin.socket
         Docs: man:libvirtd(8)
               https://libvirt.org/
    Main PID: 10117 (libvirtd)
       Tasks: 23 (limit: 32768)
          CPU: 1.104s
    CGroup: /system.slice/libvirtd.service
            └─10117 /usr/sbin/libvirtd --timeout 120
              └─10269 /usr/sbin/dnsmasq --conf-file=/var/lib/libvirt/dnsmasq/default.conf --l
                └─10270 /usr/sbin/dnsmasq --conf-file=/var/lib/libvirt/dnsmasq/default.conf --l
```

lines 1-16/16 (END)


```
# File: config.yaml
# Desc: configutarion file
---
global:
  host1_username: root
cases:
- tt_members: student_1
  host1_ip: 192.168.1.201
  host1_password: secret_1
- tt_members: student_2
  host1_ip: 192.168.1.202
  host1_password: secret_2
- tt_members: student_3
  host1_ip: 127.0.0.1
  host1_password: secret_3
```

```
# File: start.rb
# Desc: test definitions
group "Remote host" do
  target "Create user root"
  run "id root", on: :host1
  expect ["uid=", "(root)", "gid="]

  target "Delete user vader"
  run "id vader", on: :host1
  expect_fail
end

play do
  show
  export
end
```

Ejemplos y documentación

- Documentación: <https://github.com/teuton-software/teuton/examples>
- Repo de ejemplos: <https://github.com/dvarrui/teuton-tests>
- Tutorial: *Crear un test para una práctica Nginx apoyándonos en la IA.* <https://github.com/dvarrui/charlas/blob/main/teuton/nginx/docs/index.md>

Learn

Learn how write your own Teuton tests:

1. [Creating NEW test](#)
2. [Evaluating TARGET](#)
3. [Checking REMOTE HOSTS](#)
4. [Reading CONFIG file](#)
5. [Using several files](#)
6. [CHECK test syntax](#)
7. [Target WEIGHT](#)
8. [UNIQUE values](#)
9. [SEND report copies to remote hosts](#)
10. [DEBUG results](#)
11. [Export other FORMATS](#)
12. [PRESERVE old reports](#)
13. [Hide FEEDBACK from reports](#)
14. [MOODLE](#)
15. [Build README from test](#)
16. [INCLUDE more configuration files](#)
17. [ALIAS](#)
18. [LOG messages](#)

```
expect ["obiwan", "kenobi"]  
expect_first "episode"  
expect_last "the end"  
expect_one "rogue"  
expect_one ["obiwan","kenobi"]  
expect /Obiwan|obi-wan/
```

```
expect_none "vader"  
expect_none ["darth", "vader"]  
expect_none  
expect_nothing
```

```
expect_ok  
expect_fail  
expect_exit NUMBER  
expect_exit MIN..MAX
```

```
expect_sequence do  
  find "A"  
  find "B"  
  find "C"  
end
```

```
expect_sequence do  
  find "A"  
  next_to "B"  
  next_to "C"  
end
```

```
expect_sequence do  
  find "A"  
  next_to "B"  
  ignore 2  
  next_to "C"  
end
```

Ejemplo sencillo

Create your test file:

```
# File: examples/02-target/start.rb
group "Learn about targets" do
  target "Create user obiwan", weight: 2
  run "id obiwan"
  expect ["uid=", "(obiwan)", "gid="]

  target "Delete user vader"
  run "id vader"
  expect_fail
end

start do
  show
  export
end
```

Usage: Run test with `teuton run TESTPATH .`

```
$ teuton run examples/02-target
-----
Started at 2025-12-01 18:14:44 +0000
F.
Finished in 0.005 seconds
-----
```

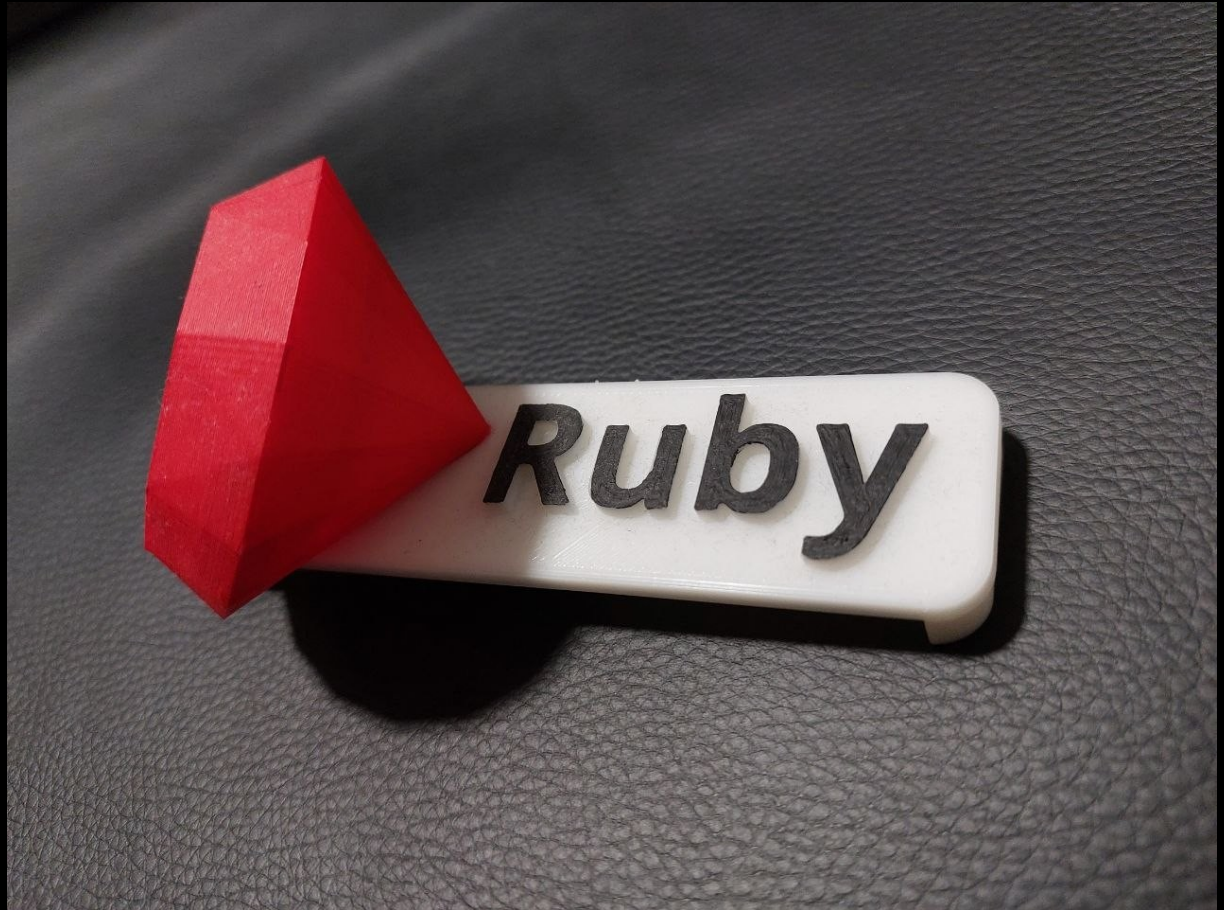
CASE RESULTS

+	-----+	-----+	-----+	-----+	
	CASE	MEMBERS	GRADE	STATE	
	01	anonymous	33.0	?	
+	-----+	-----+	-----+	-----+	

1	CONFIGURATION	21	Description : Delete user vader
2	+-----+-----+	22	Command : id vader
3	tt_members VALUE	23	Output : id: 'vader': no existe ese usuario
4	tt_sequence false	24	Duration : 0.002 (local)
5	tt_skip false	25	Alterations : Read exit code
6	tt_testname 02-target	26	Expected : Greater than 0
7	+-----+-----+	27	Result : 1
8		28	
9		29	RESULTS
10	GROUPS	30	+-----+-----+
11	- Learn about targets	31	unique_fault 0
12	01 (0.0/2.0)	32	case_id 01
13	Description : Create user obiwan	33	start_time 2025-11-12 20:15:59 +0000
14	Command : id obiwan	34	finish_time 2025-11-12 20:15:59 +0000
15	Output : id: 'obiwan': no existe	35	duration 0.004399074
16	Duration : 0.002 (local)	36	max_weight 3.0
17	Alterations : find(uid=) & find((obiwan	37	good_weight 1.0
18	Expected : Greater than 0	38	fail_weight 2.0
19	Result : 0	39	fail_counter 1
20	02 (1.0/1.0)	40	grade 33
21	Description : Delete user vader	41	+-----+-----+

DEMO!

demo/v03.custom



teuton-get: Gestor de repositorios de tests

- Instalar Ruby
- `gem install teuton-get`, instalar la gema

```
$ teuton-get help
```

```
Commands:
```

<code>teuton-get create-info DIRPATH</code>	<code># Create info data for Teuton test</code>
<code>teuton-get create-repo</code>	<code># Create repo into current directory</code>
<code>teuton-get help [COMMAND]</code>	<code># Describe available commands or one specific command</code>
<code>teuton-get init</code>	<code># Create ini config file</code>
<code>teuton-get pull TESTID [DIRECTORY]</code>	<code># Download Teuton test</code>
<code>teuton-get refresh</code>	<code># Synchronize list of available tests.</code>
<code>teuton-get repos</code>	<code># Show repo list</code>
<code>teuton-get search [REPONAME:]FILTER</code>	<code># Search Teuton test with FILTER</code>
<code>teuton-get show TESTID FILTER</code>	<code># Show info data for Teuton test</code>
<code>teuton-get tree</code>	<code># Print a tree of all available commands</code>
<code>teuton-get version</code>	<code># Show the program version</code>

ECOSISTEMA: Repositorios y proyectos

github.com/teuton-software/teuton

Teuton

github.com/teuton-software/teuton-get

Teuton get

github.com/dvarrui/teuton-tests

Repositorio de tests

EN PROCESO

github.com/dvarrui/teuton-book

Documentación (ES)

github.com/dvarrui/teuton-panel

teuton-panel

Roadmap

1. Documentación (teuton-book)
2. Difusión (Charlas, eventos, comic, game)
3. Teuton-panel
4. Feedback
5. Ruby way...

github.com/dvarrui/charlas/teuton/rubysur.2608

¡Muchas gracias!

Preguntas...



github: @dvarrui
telegram: @dvarrui
email: dvarrui@proton.me